Recursion Backtracking Study Guide

What is Recursion?

Recursion is a programming technique where a method calls itself to solve a problem. Every recursive method has two main components:

1. **Base Case**: The condition that stops the recursion
2. **Recursive Case**: The method calling itself with modified parameters

Basic Recursion Example

```java
public static int factorial(int n) {
    // Base case
    if (n <= 1) {
        return 1;
    }
    // Recursive case
    return n * factorial(n - 1);
}
```

What is Backtracking?

Backtracking is an algorithmic approach that builds solutions incrementally and abandons candidates ("backtracks") when they cannot lead to a valid solution. It's essentially a trial-and-error approach.

Key Characteristics of Backtracking:

- Try all possible solutions
- If a solution doesn't work, undo the last step and try another
- Continue until all possibilities are exhausted or a solution is found

Recursion with Backtracking

When combined, recursion and backtracking create a powerful problem-solving technique. The recursive structure explores all possibilities, while backtracking ensures we don't get stuck in invalid paths.

General Template for Backtracking

```java
public static boolean backtrack(parameters) {
    // Base case - check if solution is complete
    if (isComplete()) {
        return true; // or process solution
    }

    // Try all possible choices
    for (each possible choice) {
        // Make the choice
        makeChoice();

        // Recursively explore
        if (backtrack(modified_parameters)) {
            return true; // Solution found
        }

        // Backtrack - undo the choice
        undoChoice();
    }

    return false; // No solution found
}
```

Practice Questions on Recursion Backtracking

1. N-Queens Problem

The N-Queens problem asks to place N queens on an N×N chessboard such that no two queens attack each other.

java

```java
public class NQueens {
    private static boolean isSafe(int[][] board, int row, int col, int n) {
        // Check column
        for (int i = 0; i < row; i++) {
            if (board[i][col] == 1) {
                return false;
            }
        }

        // Check upper left diagonal
        for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--) {
            if (board[i][j] == 1) {
                return false;
            }
        }

        // Check upper right diagonal
        for (int i = row - 1, j = col + 1; i >= 0 && j < n; i--, j++) {
            if (board[i][j] == 1) {
                return false;
            }
        }

        return true;
    }

    private static boolean solveNQueens(int[][] board, int row, int n) {
        // Base case: all queens placed
        if (row >= n) {
            return true;
        }

        // Try placing queen in each column of current row
        for (int col = 0; col < n; col++) {
            if (isSafe(board, row, col, n)) {
                // Place queen
                board[row][col] = 1;

                // Recursively place remaining queens
                if (solveNQueens(board, row + 1, n)) {
                    return true;
                }
```

```
                // Backtrack
                board[row][col] = 0;
            }
        }

        return false;
    }
}
```

2. Sudoku Solver

Solve a 9×9 Sudoku puzzle using backtracking.

java

```java
public class SudokuSolver {
    private static boolean isSafe(int[][] board, int row, int col, int num) {
        // Check row
        for (int x = 0; x < 9; x++) {
            if (board[row][x] == num) {
                return false;
            }
        }

        // Check column
        for (int x = 0; x < 9; x++) {
            if (board[x][col] == num) {
                return false;
            }
        }

        // Check 3x3 box
        int startRow = row - row % 3;
        int startCol = col - col % 3;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (board[i + startRow][j + startCol] == num) {
                    return false;
                }
            }
        }

        return true;
    }

    private static boolean solveSudoku(int[][] board) {
        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                if (board[row][col] == 0) {
                    for (int num = 1; num <= 9; num++) {
                        if (isSafe(board, row, col, num)) {
                            // Place number
                            board[row][col] = num;

                            // Recursively solve
                            if (solveSudoku(board)) {
                                return true;
                            }
```

```
                        // Backtrack
                        board[row][col] = 0;
                    }
                }
                return false;
            }
        }
    }
    return true;
  }
}
```

3. Maze Path Finding

Find a path from start to end in a maze using backtracking.

```java
public class MazeSolver {
    private static boolean isValid(int[][] maze, int x, int y, int n, int m) {
        return (x >= 0 && x < n && y >= 0 && y < m && maze[x][y] == 1);
    }

    private static boolean solveMaze(int[][] maze, int x, int y, int[][] solution, int n, int m
        // Base case: reached destination
        if (x == n - 1 && y == m - 1) {
            solution[x][y] = 1;
            return true;
        }

        // Check if current position is valid
        if (isValid(maze, x, y, n, m)) {
            // Mark current cell as part of solution
            solution[x][y] = 1;

            // Move right
            if (solveMaze(maze, x, y + 1, solution, n, m)) {
                return true;
            }

            // Move down
            if (solveMaze(maze, x + 1, y, solution, n, m)) {
                return true;
            }

            // Backtrack
            solution[x][y] = 0;
            return false;
        }

        return false;
    }
}
```

4. Generate All Permutations

Generate all permutations of a given string using backtracking.

```java
public class Permutations {
    private static void generatePermutations(String str, String current, boolean[] used) {
        // Base case: permutation complete
        if (current.length() == str.length()) {
            System.out.println(current);
            return;
        }

        // Try each unused character
        for (int i = 0; i < str.length(); i++) {
            if (!used[i]) {
                // Choose character
                used[i] = true;
                current += str.charAt(i);

                // Recursively generate remaining
                generatePermutations(str, current, used);

                // Backtrack
                used[i] = false;
                current = current.substring(0, current.length() - 1);
            }
        }
    }

    public static void printPermutations(String str) {
        boolean[] used = new boolean[str.length()];
        generatePermutations(str, "", used);
    }
}
```

5. Subset Sum Problem

Find if there's a subset with a given sum using backtracking.

```java
public class SubsetSum {
    private static boolean hasSubsetSum(int[] arr, int index, int currentSum, int targetSum) {
        // Base case: target sum reached
        if (currentSum == targetSum) {
            return true;
        }

        // Base case: end of array or sum exceeded
        if (index >= arr.length || currentSum > targetSum) {
            return false;
        }

        // Include current element
        if (hasSubsetSum(arr, index + 1, currentSum + arr[index], targetSum)) {
            return true;
        }

        // Exclude current element
        if (hasSubsetSum(arr, index + 1, currentSum, targetSum)) {
            return true;
        }

        return false;
    }

    public static boolean canSum(int[] arr, int targetSum) {
        return hasSubsetSum(arr, 0, 0, targetSum);
    }
}
```

Key Points to Remember

1. **Always define a clear base case** to prevent infinite recursion
2. **Make progress towards the base case** in each recursive call
3. **Backtrack properly** by undoing changes when a path doesn't work
4. **Check constraints** before making recursive calls to improve efficiency
5. **Use appropriate data structures** to track the current state

Common Backtracking Patterns

Pattern 1: Decision Trees

- At each step, make a choice from available options
- Recursively explore the consequences
- Backtrack if the choice leads to failure

Pattern 2: Constraint Satisfaction

- Place elements one by one
- Check if placement violates constraints
- Backtrack if constraints are violated

Pattern 3: Optimization Problems

- Generate all possible solutions
- Keep track of the best solution found
- Prune branches that cannot lead to better solutions

Quiz Questions

1. What is the most important component that prevents infinite recursion in backtracking? a) Recursive case b) Base case c) Loop condition d) Return statement

Answer: b) Base case

2. In the N-Queens problem, what do we check to ensure a queen placement is safe? a) Only the current row b) Only the current column c) Row, column, and both diagonals d) Only the diagonals

Answer: c) Row, column, and both diagonals

3. What happens during the "backtrack" step in a backtracking algorithm? a) Move to the next possible choice b) Undo the last choice made c) Check if solution is complete d) Print the current solution

Answer: b) Undo the last choice made

4. Which of the following problems is best solved using backtracking? a) Finding the maximum element in an array b) Sorting an array c) Generating all permutations of a string d) Calculating factorial

Answer: c) Generating all permutations of a string

5. In backtracking, when do we typically return true from a recursive function? a) When we start the recursion b) When we find a complete valid solution c) When we encounter an invalid state d) When we finish exploring all possibilities

Answer: b) When we find a complete valid solution

Practice Coding Questions

1. Rat in a Maze Write a program to find a path for a rat to move from the top-left corner to the bottom-right corner of a maze. The rat can only move right or down, and some cells are blocked (marked as 0). Return true if a path exists, false otherwise.

2. Word Search Given a 2D board of characters and a word, determine if the word exists in the grid. The word can be constructed from letters of adjacent cells (horizontally or vertically). Each cell can only be used once in a single word.

3. Combination Sum Given an array of distinct integers and a target sum, find all unique combinations where the numbers sum to the target. The same number can be used multiple times in a combination.

4. Generate Parentheses Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses. For example, given n = 3, the output should be: ["((()))", "(()())", "(())()", "()(())", "()()()"]

5. Knight's Tour Given a chessboard of size N×N, find a sequence of moves for a knight such that it visits every square exactly once. Start from position (0,0) and return the path as a 2D array showing the order of moves.